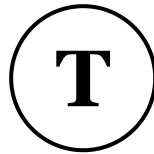


INTERNATIONAL ISLAMIC UNIVERSITY CHITTAGONG

Department of Computer Science and Engineering



PROJECT REPORT

TUITIONIFY

*A Privacy-Aware Tuition Marketplace
for the Bangladeshi Education Context*

Course Title: _____

Course Code: _____

Semester: _____

Submitted by

Iftaher Sayem

Student ID: _____

Submitted to

DECLARATION

Department of CSE, IIUC

I hereby declare that this project report titled **“Tuitionify — A Privacy-Aware Tuition Marketplace for the Bangladeshi Education Context”** is the result of my own work carried out in the Department of Computer Science and Engineering, International Islamic University Chittagong.
Date of Submission: _____

The design, implementation, testing and security analysis presented in this report were performed by me. Any external libraries, services and sources that were used are acknowledged in the text and listed in the References. This work has not been submitted elsewhere for the award of any other degree or diploma.

I further declare that the security findings reported in Chapter 7 were identified during a review of my own codebase, that the fixes described there were implemented and verified by me, and that the limitations which remain unresolved have been stated openly rather than omitted.

Iftaher Sayem

Student ID: _____

Department of CSE, IIUC

Approved by

Supervisor

Department of CSE, IIUC

ACKNOWLEDGEMENTS

All praise is due to Allah, by whose mercy this work was brought to completion.

I am grateful to my supervisor for the guidance and patience that shaped this project, and for the questions that repeatedly sent me back to the code with a clearer idea of what I was actually building.

I thank the faculty of the Department of Computer Science and Engineering at IIUC, whose teaching in database systems, software engineering and web technologies provided the foundation this project rests on.

I am indebted to the classmates who tested early builds and reported what confused them. Several decisions in the final system — the review-eligibility notice, the dashboard prompts for a missing phone number, the visible reason a contact detail is hidden — exist because a real user was puzzled first.

Finally, I thank my family for their support and encouragement throughout.

Iftaher Sayem

August 2026

TABLE OF CONTENTS

CHAPTER 1 — INTRODUCTION

- 1.1 Project Overview
- 1.2 Problem Statement
- 1.3 Objectives
- 1.4 Scope
- 1.5 Report Organisation

CHAPTER 2 — BACKGROUND AND LITERATURE REVIEW

- 2.1 The Private Tuition Market in Bangladesh
- 2.2 Existing Solutions and Their Gaps
- 2.3 Key Concepts Applied

CHAPTER 3 — METHODOLOGY

- 3.1 Process Model
- 3.2 Requirements Analysis
- 3.3 Technology Choices and Justification
- 3.4 Tools

CHAPTER 4 — SYSTEM ANALYSIS AND DESIGN

- 4.1 Actors and Use Cases
- 4.2 System Architecture
- 4.3 Database Design
- 4.4 Trust and Safety Rules
- 4.5 End-to-End Workflow

CHAPTER 5 — IMPLEMENTATION

- 5.1 Frontend
- 5.2 Backend and API
- 5.3 Security Hardening Applied
- 5.4 Project Structure

CHAPTER 6 — TESTING

6.1 Testing Strategy

6.2 Unit Test Coverage

6.3 Mutation Verification

6.4 End-to-End Verification

CHAPTER 7 — SECURITY ANALYSIS

7.1 Security Posture

7.2 Findings and Their Resolution

7.3 Accepted Limitations

CHAPTER 8 — RESULTS AND DISCUSSION

8.1 Delivered Features

8.2 API Summary

8.3 Discussion

CHAPTER 9 — CONCLUSION AND FUTURE WORK

9.1 Conclusion

9.2 Future Work

ABSTRACT

Private tuition is a deeply embedded part of secondary and higher-secondary education in Bangladesh, yet finding a trustworthy tutor still depends on word of mouth, printed leaflets, and scattered Facebook groups. This report presents **Tuitionify**, a full-stack web application that brings the process online for the International Islamic University Chittagong (IIUC) community. Tuitionify connects university-student tutors with guardians and students through a structured hiring loop — posting, filtering, applying, accepting, and reviewing — while keeping personal contact details private until a relationship is established.

The system is built as a React single-page application on the frontend, a Node.js and Express REST API on the backend, MongoDB Atlas for persistence, and Firebase for authentication. It is shaped around the Bangladeshi context in concrete ways: salaries are quoted in BDT and bounded to a realistic range; the class system, curriculum subjects and Chittagong areas are enforced as validated option lists; and mobile numbers are normalised and validated as real Bangladeshi numbers, because hiring ultimately ends in a phone call.

Because trust is the product, the design treats safety as a core requirement rather than an afterthought. Reviews may only be written by seekers who have actually engaged a tutor; reviews and reports require a confirmed email address; restricted accounts are refused everywhere, including on public pages; and contact details are removed from API responses server-side rather than merely hidden in the interface. A security review of the completed system identified ten findings, of which eight were resolved, one was patched at the dependency level, and one was assessed and deliberately not changed after inspection showed it was not reachable. A Vitest suite of seventy-two tests covers the rules that matter most, and was itself verified by deliberately removing protections and confirming the tests fail.

Keywords: tuition marketplace, trust and safety, review integrity, privacy-gated contact sharing, Bangladeshi education system, MERN, Firebase, MongoDB, Express, React, Vitest

CHAPTER 1 — INTRODUCTION

1.1 Project Overview

Tuitionify is a web-based tuition marketplace that connects tutors with guardians and students. It is scoped to the IIUC community, where the two sides of the market naturally coexist: university students who want part-time teaching income, and families in and around Chittagong who want a qualified tutor for a school-going child.

The application supports two user roles. A **tutor** builds a public profile stating subjects, class levels, preferred areas, expected salary and qualifications, then browses and applies to posted tuitions. A **seeker** — a guardian or a student — posts a tuition requirement, reviews the tutors who apply, and accepts one. A third role, **admin**, exists for moderation: verifying tutors, handling reports, and restricting accounts that misbehave.

What distinguishes the system from a simple listings board is that it models the *hiring relationship*, not just the advertisement. An application has a state. Accepting an applicant closes the tuition. Contact details become visible only once one party has agreed to share them. A review can only be written by someone the tutor actually agreed to work with. These rules are enforced on the server, where a user cannot bypass them.

1.2 Problem Statement

The existing process for arranging private tuition in Bangladesh suffers from several concrete problems.

Discovery is inefficient. Guardians rely on personal referrals, leaflets stapled to notice boards, and Facebook groups where posts scroll out of view within hours. A guardian in one part of Chittagong has no reliable way to find a suitable tutor in the same area.

There is no verifiable reputation. A tutor's claim about their own experience cannot be checked. Existing platforms that do show ratings rarely restrict who may rate, so a rating can be inflated by friends or by throwaway accounts. A rating that anyone can write is not evidence of anything.

Contact information is exposed prematurely. When a phone number is published openly, it is harvested and misused. Guardians are particularly reluctant to publish a number attached to a post that also mentions a child and a home area.

Generic platforms do not fit the local context. International tutoring sites assume different currencies, grade systems, and subject names. A guardian looking for a Class 9 Higher Math tutor in Chawkbazar is not served by a platform built around a different curriculum.

1.3 Objectives

The project set out to achieve the following.

1. To design and implement a functioning two-sided marketplace covering the complete hiring loop: post, discover, apply, accept, and review.
2. To keep personal contact details private by default, and to share them only after a mutual step has taken place.
3. To ensure that ratings reflect real engagements, by permitting reviews only from seekers who have demonstrably engaged the tutor.
4. To fit the Bangladeshi education context precisely, through validated subject, class-level and area lists, BDT salary bounds, and Bangladeshi mobile number handling.
5. To provide moderation capabilities so that misuse can be reported and acted upon.
6. To subject the finished system to a security review, resolve what is resolvable, and state the remaining limitations honestly.
7. To verify the rules that carry the most risk with an automated test suite.

1.4 Scope

Within scope. Role-based registration and authentication; tutor profiles with photo upload; tuition posting with structured requirements; filtered search over both tutors and tuitions; the application lifecycle including withdrawal; a contact-request mechanism; engagement-gated reviews with tutor replies; bookmarking of both tuitions and tutors; reporting and admin moderation; an admin analytics dashboard; and a light and dark theme.

Outside scope. In-app messaging, online payment or salary escrow, video lessons, scheduling and calendar management, a mobile application, and identity verification against national documents. Chapter 9 discusses which of these are the natural next steps.

1.5 Report Organisation

Chapter 2 examines the market context and the gaps in existing solutions. Chapter 3 sets out the development methodology, the requirements, and the reasoning behind each technology choice. Chapter 4 presents the analysis and design, including the database schema and the trust rules.

Chapter 5 describes the implementation. Chapter 6 covers testing, including a mutation exercise used to check that the tests genuinely detect regressions. Chapter 7 reports the security review in full. Chapter 8 presents results, and Chapter 9 concludes with future work.

CHAPTER 2 — BACKGROUND AND LITERATURE REVIEW

2.1 The Private Tuition Market in Bangladesh

Private tuition occupies an unusual position in Bangladeshi education. It is not a marginal supplement but a widely assumed part of preparation for public examinations, particularly at the SSC and HSC stages. Households across a broad income range treat tuition costs as a routine expense, and university students form a large and willing supply of tutors, for whom teaching is one of the few flexible sources of income compatible with a class timetable.

The market is therefore large, active, and almost entirely informal. Arrangements are made verbally, terms are agreed without documentation, and reputation travels by word of mouth within a neighbourhood or a circle of relatives. This informality is efficient within a small social network and fails completely outside it. A guardian who has recently moved to a new area has no network to draw on.

Three features of the local context bear directly on the design of any software intended to serve this market. First, the currency and salary expectations are specific: monthly rates quoted in BDT, typically in the low thousands, and a system that accepts an unbounded number invites both typos and unrealistic postings. Second, the academic structure is specific: classes numbered through the school system, then HSC, then admission preparation, with subject names following the national curriculum. Third, and most practically, the transaction concludes by telephone. A platform may mediate discovery, but the hiring conversation happens on a mobile call, which makes a valid mobile number a functional requirement rather than an optional profile field.

2.2 Existing Solutions and Their Gaps

Several categories of existing solution were examined before design began.

Social media groups. Facebook groups dedicated to tuition in particular cities are the dominant tool in practice. Their advantage is reach and zero setup cost. Their weaknesses are structural: posts are unstructured free text, so filtering by class, subject or area is impossible; content is ordered by recency and social engagement rather than relevance; there is no notion of an application, so a post attracts a disorganised flood of comments; and phone numbers are posted publicly, exposing both parties.

Local classifieds platforms. General-purpose classified sites carry tuition listings. They provide structure for search but treat tuition as a commodity listing. There is no concept of a tutor profile with a verifiable history, no application state, and no reputation mechanism tied to actual engagements.

International tutoring marketplaces. Established global platforms solve the reputation and scheduling problems well, and some implement contact gating and payment escrow. They are, however, built around different curricula, currencies and payment instruments. Their subject taxonomies and grade systems do not map onto the Bangladeshi system, and their payment rails are largely unavailable locally.

A recurring weakness across all three categories deserves emphasis, because it motivated one of the central design decisions in this project. Where ratings exist at all, they are rarely tied to a completed transaction. A platform that allows any registered account to rate any tutor produces a rating that measures social reach rather than teaching quality. Since a rating is precisely the signal a guardian most wants to rely on, an unverified rating is worse than no rating: it invites misplaced confidence.

2.3 Key Concepts Applied

Two-sided marketplace design. The system must serve two distinct user groups whose value to each other depends on the presence of the other side. This motivates giving both sides a symmetric discovery path: seekers browse tutors, and tutors browse tuitions, so neither side must wait passively.

Progressive disclosure of personal data. Rather than treating contact information as either public or private, the system treats it as conditionally visible. The condition is a mutual step: an accepted application, or an explicitly approved contact request. This principle is applied server-side, so the data never reaches a client that is not entitled to it.

Verified reputation. Reviews are gated on evidence of a real engagement. The design requirement is that the gate must depend on an action taken by the party being reviewed, so that the reviewed party cannot be targeted by a reviewer acting alone.

Defence in depth. Authorisation is checked at multiple layers rather than at a single gate: token verification, account loading, restriction checks, role checks, resource ownership checks, and field-level whitelisting on write operations. Chapter 5 details how these compose.

Principle of least privilege on writes. Every write endpoint accepts an explicit whitelist of fields rather than assigning a request body wholesale, so that sensitive fields such as verification

status cannot be set by a client.

CHAPTER 3 — METHODOLOGY

3.1 Process Model

The project followed an **incremental and iterative** process. A minimum working system was built first — authentication, the two roles, tuition posting, and applications — and further capability was layered on in successive increments, each ending in a runnable state.

This suited the project for two reasons. The requirements were not fully known at the outset, since several rules only became apparent once the system could actually be used; and the trust rules in particular emerged from reasoning about how a working marketplace could be abused, which is difficult to do productively in the abstract.

Increment	Delivered
1	Firebase authentication, role selection, user profiles
2	Tuition posting, listing, and filtering
3	Application lifecycle, role-based dashboards
4	Reviews and ratings, tutor directory
5	Privacy-gated contact requests
6	Reporting, admin moderation, verification badge
7	Dark mode, pagination, search, bookmarks, photo upload
8	Analytics, recommendations, review replies, application timeline
9	Security review; hardening; trust rules; test suite

Table 3.1: Development increments

The final increment deserves comment as a matter of methodology. Rather than declaring the project finished once the features worked, the completed system was deliberately reviewed as an adversary would examine it, asking what a motivated user could do that the design did not intend. That exercise produced the most consequential change in the project — the review-eligibility rule — which no feature specification had called for. This is recorded here because it shaped the outcome more than any single feature.

3.2 Requirements Analysis

3.2.1 Functional Requirements

ID	Requirement	Actor
FR-01	Register and authenticate by email/password or Google	All
FR-02	Select a role at registration	All
FR-03	Create and edit a tutor profile	Tutor
FR-04	Upload a profile photograph	All
FR-05	Post a tuition requirement	Seeker
FR-06	Edit and delete an own posting	Seeker
FR-07	Browse and filter tuitions	All
FR-08	Browse and filter tutors	All
FR-09	Apply to an open tuition	Tutor
FR-10	Withdraw a pending application	Tutor
FR-11	View applicants and accept or reject	Seeker
FR-12	Request a tutor's contact details	Seeker
FR-13	Approve or decline a contact request	Tutor
FR-14	Review a tutor after engaging them	Seeker
FR-15	Reply to a received review	Tutor
FR-16	Bookmark tuitions and tutors	All
FR-17	Report a profile or posting	All
FR-18	Grant or revoke the verified badge	Admin
FR-19	Restrict or unrestrict an account	Admin
FR-20	Process the report queue	Admin
FR-21	View platform analytics	Admin
FR-22	Receive tuition recommendations	Tutor

Table 3.2: Functional requirements

3.2.2 Non-Functional Requirements

ID	Requirement	How it is met
NFR-01	Contact details must not be exposed without consent	Fields deleted server-side; conditional disclosure
NFR-02	Ratings must reflect real engagements	Engagement rule enforced on the API
NFR-03	Passwords must never be stored by the application	Delegated to Firebase Authentication
NFR-04	Abuse must be rate-limited	Per-user and per-IP limits on all write routes
NFR-05	Input must be validated before persistence	Whitelists, enums, bounds and length caps
NFR-06	The interface must work on a mobile handset	Mobile-first responsive layout
NFR-07	Listings must not return unbounded responses	Pagination on every list endpoint
NFR-08	Errors must not disclose internal detail	Central handler; messages suppressed in production
NFR-09	Critical rules must be regression-protected	Automated test suite

Table 3.3: Non-functional requirements

3.3 Technology Choices and Justification

Layer	Chosen	Justification
Frontend	React 18 + Vite 5	Component reuse across many similar listing views; Vite gives fast rebuilds and per-route code splitting
Routing	React Router 6	Declarative guards, and redirect-with-state so a user returns to the page they wanted after login
Styling	Tailwind CSS 3	Consistent spacing and colour without a separate stylesheet; class-based dark mode
HTTP	Axios	A single request interceptor attaches the auth token application-wide
Backend	Node.js + Express 4	One language across the stack; middleware composition maps naturally onto layered authorisation
Database	MongoDB Atlas + Mongoose 8	Document model suits a schema that evolved each increment; Mongoose validators enforce rules at the persistence layer
Authentication	Firebase Auth + Admin SDK	Provides email verification, password reset and Google sign-in without storing credentials locally
Images	imgbb via server proxy	Free hosting; proxying keeps the API key off the client
Testing	Vitest 2	ESM-native, fast, and runs without a database
Hosting	Vercel	Free tier, deploys on push, serverless functions for the API

Table 3.4: Technology choices

Two choices warrant fuller justification.

Delegating authentication to Firebase. Implementing password storage, reset flows and email verification correctly is a significant undertaking with a high penalty for error. Delegating it removes an entire class of vulnerability: since the application never receives a password, there is no credential hash in the database to be leaked. The cost is a dependency on an external provider and the need to verify tokens on every request.

MongoDB rather than a relational database. The data is naturally document-shaped, particularly the user record, where a single collection serves two roles with different populated fields. The relationships are shallow — no query requires deep joins. The trade-off is that referential integrity is not enforced by the database, so cascade deletion is handled explicitly in application code, as described in Chapter 5.

3.4 Tools

Purpose	Tool
Editor	Visual Studio Code
Version control	Git and GitHub
API testing	Thunder Client; direct fetch scripts
Database inspection	MongoDB Compass; Atlas web console
Dependency auditing	<code>npm audit</code>
Automated testing	Vitest
Deployment	Vercel CLI and dashboard

Table 3.5: Development tools

CHAPTER 4 — SYSTEM ANALYSIS AND DESIGN

4.1 Actors and Use Cases

Actor	Description
Visitor	Unauthenticated. May browse tuitions and tutor profiles, without contact details.
Tutor	Authenticated university student. Maintains a profile, applies to tuitions, answers contact requests, replies to reviews.
Seeker	Authenticated guardian or student. Posts tuitions, decides on applicants, requests contact, writes reviews when eligible.
Admin	Identified by a configured email address. Verifies tutors, restricts accounts, processes reports, views analytics.

Table 4.1: System actors

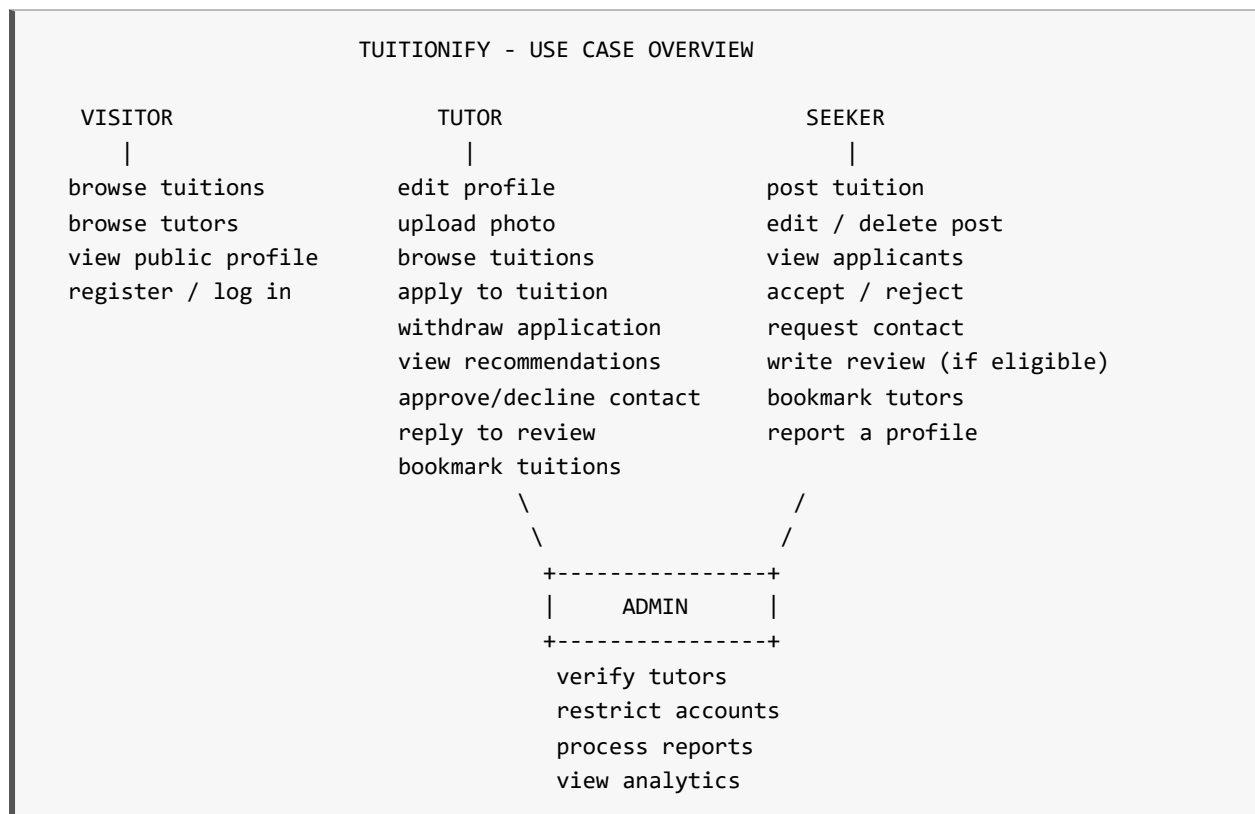


Figure 4.1: Use case overview

4.2 System Architecture

The system uses a three-tier architecture. The presentation tier is a React single-page application; the application tier is an Express REST API; the data tier is MongoDB Atlas. Authentication is provided by Firebase, and image hosting by imgbb, both as external services.

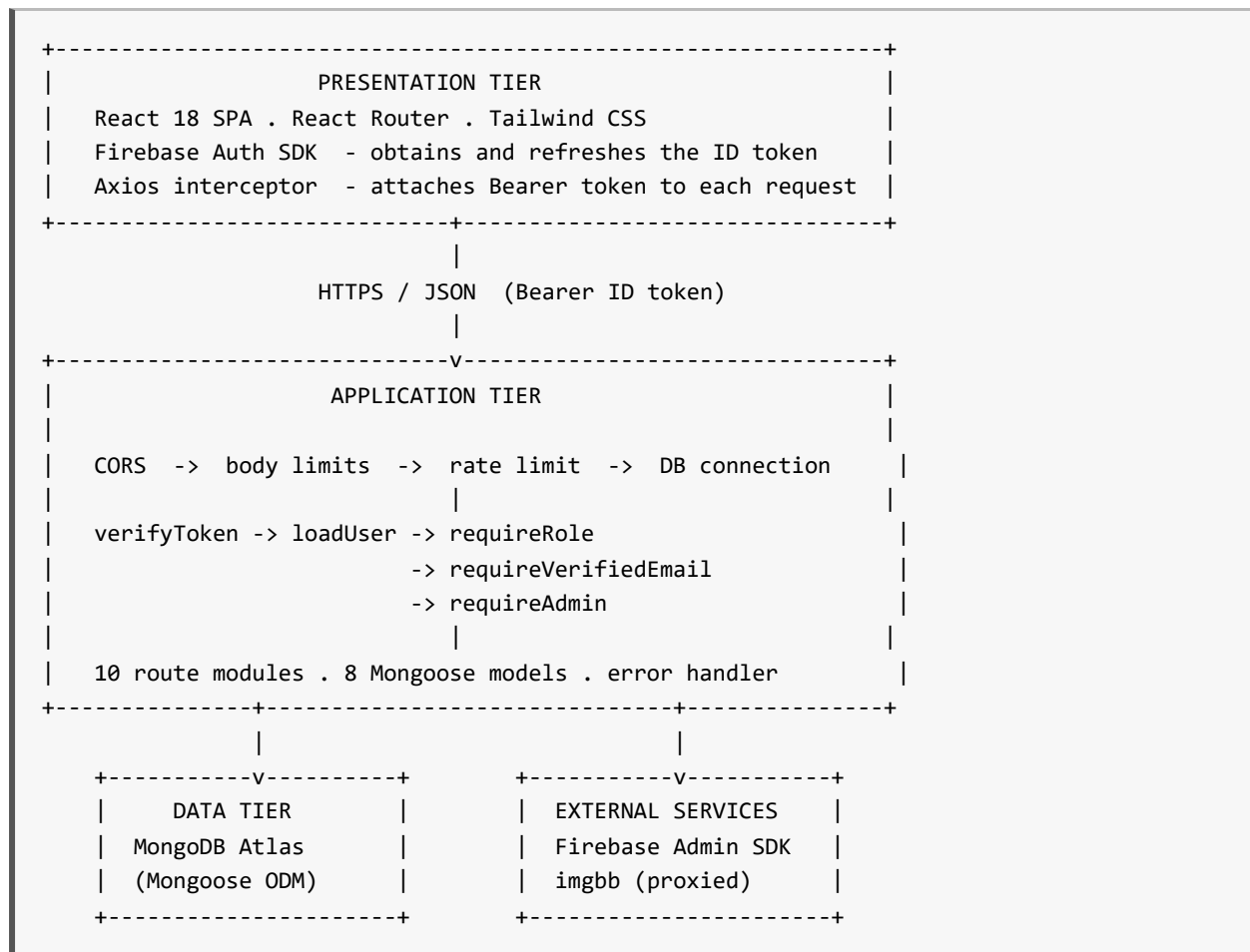
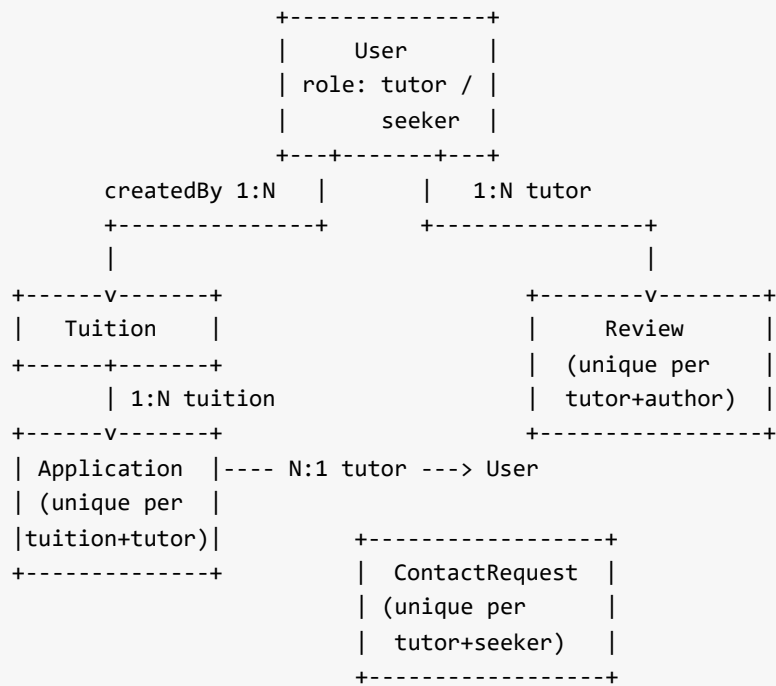


Figure 4.2: Three-tier architecture

The client and the API are deployed as two independent Vercel projects from a single repository. This has one design consequence worth recording: the shared option lists for subjects, class levels and areas exist in both halves of the codebase and cannot be shared by import, because the two deployments have separate root directories and no common build step. The duplication is deliberate and is documented in both files, since silent drift between them would cause the API to reject values the interface offers.

4.3 Database Design

Eight collections. All carry automatic `createdAt` and `updatedAt` timestamps.



Bookmark: user -> tuition (unique)
TutorBookmark: user -> tutor (unique)
Report: reporter -> targetId (targetType: user | tuition)

Figure 4.3: Entity relationships

4.3.1 The User Collection

A single collection serves both roles, with tutor-specific fields left empty for seekers.

Field	Type	Constraints
firebaseUid	String	Required, unique, indexed
name	String	Required, max 100
email	String	Required; taken from the verified token only
role	String	tutor or seeker
phone	String	Normalised Bangladeshi mobile
photo	String	Must be an http(s) URL
gender	String	male, female or empty
university, department	String	Max 150 each
subjects	[String]	Max 50, each from the subject list
classLevels	[String]	Max 50, each from the class list
preferredAreas	[String]	Max 50, each from the area list
expectedSalary	Number	0 to 100,000 BDT
mode	String	home, online, both or empty
bio	String	Max 2,000
ratingAvg, ratingCount	Number	Computed by the server from reviews
isVerified	Boolean	Granted by an admin
emailVerified	Boolean	Mirrors the Firebase token claim
restricted	Boolean	Set by an admin; blocks the account

Table 4.2: User schema

4.3.2 The Tuition Collection

Field	Type	Constraints
title	String	Required, max 150
classLevel	String	Required, from the class-level list
subjects	[String]	1 to 20, each from the subject list
area	String	Required, from the area list
salary	Number	500 to 100,000 BDT per month
daysPerWeek	Number	1 to 7
mode	String	home or online
genderPreference	String	male, female or any
description	String	Max 3,000
status	String	open or closed ; server-controlled
createdBy	ObjectId	Reference to User; server-set

Table 4.3: Tuition schema

4.3.3 Remaining Collections

Collection	Purpose	Key constraint
Application	A tutor applying to a tuition, with status and audit timestamps	Unique on tuition + tutor
Review	A 1–5 rating with comment and optional tutor reply	Unique on tutor + author
ContactRequest	A seeker asking a tutor to share contact details	Unique on tutor + seeker
Report	A flagged profile or posting awaiting moderation	Target type restricted to two values
Bookmark	A saved tuition	Unique on user + tuition
TutorBookmark	A saved tutor	Unique on user + tutor

Table 4.4: Supporting collections

The unique compound indexes are load-bearing rather than incidental. They enforce, at the database level, that a tutor cannot apply twice to one tuition, that a seeker cannot review one

tutor twice, and that a bookmark cannot be duplicated. The application catches the resulting duplicate-key error and converts it into a clear message rather than a server fault.

4.4 Trust and Safety Rules

These rules are the design response to the weaknesses identified in Chapter 2. Each is enforced in the API.

4.4.1 Review Eligibility

A seeker may review a tutor only if at least one of the following holds:

1. the tutor was **accepted** on a tuition that the seeker themselves posted, or
2. the tutor **approved** a contact request from that seeker.

The design constraint is that both conditions require an affirmative act by the tutor. A seeker acting alone cannot manufacture either one, and therefore cannot manufacture standing to review.

The reason this rule matters more than its size suggests is that average rating is the primary sort key of the public tutor directory. Registration is free and unlimited, so without the rule a person could create accounts and move a competitor down the ranking or their own profile up. The unique index prevents one account from reviewing twice; it offers no protection against ten accounts each reviewing once.

```
// server/utils/engagement.js
const myTuitionIds = await Tuition.find({ createdBy: seekerId }).distinct('_id');

if (myTuitionIds.length) {
  const accepted = await Application.exists({
    tutor: tutorId,
    tuition: { $in: myTuitionIds }, // scoped to the seeker's own postings
    status: 'accepted',
  });
  if (accepted) return true;
}

return Boolean(await ContactRequest.exists({
  tutor: tutorId, seeker: seekerId, status: 'approved',
}));
```

Figure 4.4: The engagement check

4.4.2 Confirmed Email for Reviews and Reports

Reviews and reports additionally require a confirmed email address. This is what makes the previous rule expensive to circumvent at scale, since each additional account now costs a real, reachable mailbox. Posting a tuition and applying are deliberately not gated this way, as the added friction on the core marketplace loop would outweigh the benefit; both are already restricted by role and by rate limit.

4.4.3 A Valid Mobile Number Before Applying

Because hiring concludes by telephone, a tutor must have a valid Bangladeshi mobile number saved before applying. A guardian who accepts a tutor they cannot telephone has not hired anyone, and the application merely clutters their list.

Numbers are validated as 01 followed by an operator digit in the range 3–9 and eight further digits, and are normalised on write, since the same number is written many ways in practice.

Entered by the user	Stored
01712345678	01712345678
+8801712345678	01712345678
8801712345678	01712345678
017 1234 5678	01712345678
017-1234-5678	01712345678

Table 4.5: Mobile number normalisation

The profile field itself remains optional, so a tutor browsing the site is not forced through a form prematurely. The requirement is applied at the point where it becomes meaningful, and the dashboard displays a notice beforehand so that it is not discovered only at the moment of applying.

4.4.4 Restriction Applies Everywhere

A restricted account is refused on authenticated routes. On the public tutor and tuition detail pages, which accept optional authentication in order to show additional data to signed-in users, a restricted account is treated as an anonymous visitor. Without this, a restricted seeker would retain the contact visibility granted by an earlier approved request, and the restriction would be only partially effective.

4.4.5 Server-Side Removal of Personal Data

Every response path that could carry a telephone number or email address removes the field before responding, rather than relying on the interface to conceal it: tutor listings, tutor detail, tuition detail, applicant lists before acceptance, and a seeker's own contact-request list before approval. A user reading the raw API response therefore sees exactly what a user reading the rendered page sees.

4.5 End-to-End Workflow

SEEKER	SYSTEM	TUTOR
-- register (role: seeker) ----->	<--- register (role: tutor) --	
-- post tuition ----->		
	---- appears in listings ---->	
	<---- apply (phone required) -	
<-- applicant list -----		
(phones hidden)	---- marked as viewed ----->	
-- accept one ----->		
tuition closes automatically	---- outcome + phone shown -->	
<-- accepted tutor's phone ----		
===== hiring conversation takes place by telephone =====		
-- write review ----->		
(engagement verified)	---- notified of review ----->	
	<--- reply to review -----	
<-- reply visible -----		

Figure 4.5: The complete hiring workflow

CHAPTER 5 — IMPLEMENTATION

5.1 Frontend

The frontend consists of **14 pages** and **17 reusable components**. All pages are lazy-loaded, so the charting library used by the admin dashboard is downloaded only when an administrator opens that page, rather than by every visitor.

Page	Route	Access
Home	/	Public
Login	/login	Public
Register	/register	Public
CompleteProfile	/complete-profile	Authenticated
AuthAction	/auth/action	Public
Tuitions	/tuitions	Public
TuitionDetails	/tuitions/:id	Public
Tutors	/tutors	Public
TutorProfile	/tutors/:id	Public
PostTuition	/post-tuition	Seeker
EditTuition	/edit-tuition/:id	Seeker
Dashboard	/dashboard	Authenticated
Admin	/admin	Admin
NotFound	*	Public

Table 5.1: Application pages

5.1.1 State Management

State is held in two React contexts, with no external state library. `AuthContext` owns the Firebase user, the database profile, and the derived role flags. `ThemeContext` owns the light or dark preference, persisted to local storage. All other state is local to a component. At this scale the approach keeps data flow readable; a larger application would justify a dedicated store.

5.1.2 Token Handling

A single Axios interceptor attaches the authentication token to every outbound request, so no individual call site can forget it. The token is requested fresh each time rather than cached, which allows the Firebase SDK to refresh it transparently as it approaches expiry.

```
// client/src/api/client.js
api.interceptors.request.use(async (config) => {
  const user = auth.currentUser;
  if (user) {
    const token = await user.getIdToken();
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

Figure 5.1: Automatic token attachment

5.1.3 Client-Side Route Protection

The route guard handles four cases: loading, authenticated without a profile, unauthenticated, and authorised but of the wrong role. It exists to prevent a user reaching a page that would not work for them; it is not a security boundary. Every protected route is independently enforced on the server, so disabling the guard in a browser reveals nothing.

5.2 Backend and API

The API exposes **41 endpoints** across **10 route modules**, backed by **8 Mongoose models**.

Module	Endpoints	Responsibility
<code>users.js</code>	5	Registration, own profile, tutor directory and detail
<code>tuitions.js</code>	9	Posting, listing, editing, deletion, applicant list, recommendations
<code>applications.js</code>	4	Apply, list own, decide, withdraw
<code>reviews.js</code>	3	Create, list by tutor, tutor reply
<code>contact.js</code>	4	Request contact, list incoming and outgoing, decide
<code>reports.js</code>	1	Submit a report
<code>admin.js</code>	7	Moderation and analytics
<code>bookmarks.js</code>	3	Tuition bookmarks
<code>tutorBookmarks.js</code>	3	Tutor bookmarks
<code>uploads.js</code>	1	Proxied image upload
<i>plus a health check in the application entry point</i>		1

Table 5.2: API modules

5.2.1 The Middleware Chain

Middleware	Responsibility	On failure
<code>verifyToken</code>	Verifies the Bearer token via the Firebase Admin SDK	401
<code>loadUser</code>	Loads the database profile; refuses restricted accounts	404 / 403
<code>optionalAuth</code>	Attaches a user if a valid token is present; skips restricted accounts; never rejects	—
<code>requireRole</code>	Requires a specific role	403
<code>requireVerifiedEmail</code>	Requires a confirmed address; trusts the live token first	403
<code>requireAdmin</code>	Matches the account email against the configured admin list	403

Table 5.3: Authorisation middleware

The middleware composes so that each route declares exactly the conditions it requires, in order of increasing specificity.

```
// server/routes/reviews.js
router.post('/',
  verifyToken,          // is this a valid, current token?
  loadUser,             // who is it, and are they restricted?
  requireRole('seeker'), // only seekers review tutors
  requireVerifiedEmail, // a throwaway account should cost a mailbox
  rateLimit({ windowMs: 3_600_000, max: 15, name: 'review' }),
  handler,              // which then verifies engagement
);
```

Figure 5.2: Composed authorisation on the review endpoint

5.2.2 Cascade Deletion

Because MongoDB does not enforce referential integrity, deletion is handled explicitly. Removing a tuition also removes its applications, bookmarks and reports, so no orphaned documents remain.

```
await Promise.all([
  Application.deleteMany({ tuition: tuition._id }),
  Bookmark.deleteMany({ tuition: tuition._id }),
  Report.deleteMany({ targetType: 'tuition', targetId: tuition._id }),
]);
await tuition.deleteOne();
```

Figure 5.3: Cascade deletion of a tuition

5.2.3 Proxied Image Upload

Profile photographs are uploaded through the API rather than directly from the browser, because the image host requires an API key. Any value exposed to the client build is embedded in the public JavaScript bundle and can be extracted from it, so the key is held only on the server. The endpoint also re-checks the decoded size and distinguishes an invalid key from a rejected file from an upstream rate limit, so that a misconfiguration is not reported to the user as a bad photograph.

5.3 Security Hardening Applied

5.3.1 Query Input Coercion

Express parses bracket notation in a query string into an object, so a request such as `?area[$ne]=x` would introduce a database operator where a value was expected. Every filter value

is therefore coerced to a scalar before use.

```
// server/utils/sanitize.js
export function asString(value) {
  if (value === undefined || value === null) return '';
  if (Array.isArray(value)) return typeof value[0] === 'string' ? value[0] : '';
  return typeof value === 'string' ? value : '';
}
```

Figure 5.4: Forcing a query parameter to a scalar

Search terms are additionally escaped and length-capped, which prevents both a server fault from an unbalanced bracket and a denial-of-service through a pathological expression.

5.3.2 Write Whitelisting

No endpoint assigns a request body wholesale. Each copies only an explicit list of permitted fields, so that verification status, restriction status, rating aggregates, posting status and ownership cannot be set by a client.

5.3.3 Validation at the Persistence Layer

Constraints are declared on the schemas, so they hold regardless of which code path performs the write.

Field group	Constraint
Names, titles, biographies, comments	Length caps from 100 to 3,000 characters
Subjects, class levels, areas	Membership of the permitted list; maximum count
Salary	500 to 100,000 BDT per month
Days per week	1 to 7
Rating	1 to 5
Photograph URL	http or https scheme only
Mobile number	Valid Bangladeshi format after normalisation

Table 5.4: Persistence-layer constraints

Matching maximum lengths are set on the corresponding form inputs, so the browser prevents overflow before a request is sent and the server constraint acts as a backstop rather than a surprise.

5.3.4 Rate Limiting

Action	Limit	Window
Any request (backstop)	200	1 minute
Post a tuition	10	1 minute
Upload a photograph	10	1 minute
Apply to a tuition	30	1 hour
Request contact	20	1 hour
Post a review	15	1 hour
Submit a report	10	1 hour

Table 5.5: Rate limits

Authenticated requests are counted per user, anonymous requests per forwarded address. The counters are held in memory, which on a serverless platform means they are per-instance. This is sufficient to stop a single client from repeatedly calling an endpoint, but is not a strict global limit; the trade-off is recorded in Chapter 7.

5.4 Project Structure

```

tuitionify/
|-- client/                                # React application
|   |-- src/
|   |   |-- api/client.js                 # Axios instance and interceptor
|   |   |-- components/                  # 17 reusable components
|   |   |-- context/                     # AuthContext, ThemeContext
|   |   |-- data/options.js              # Subjects, classes, Chittagong areas
|   |   |-- firebase/config.js           # Firebase web SDK setup
|   |   |-- pages/                       # 14 pages
|   |   |-- App.jsx                      # Route table
|   |   +-- main.jsx
|   +-- vercel.json
|
+-- server/                                # Express API
    |-- config/
    |   |-- db.js                        # Cached database connection
    |   +-- firebase.js                  # Admin SDK initialisation
    |-- middleware/
    |   |-- auth.js                      # Six authorisation guards
    |   +-- rateLimit.js
    |-- models/                          # 8 Mongoose schemas
    |-- routes/                          # 10 modules, 41 endpoints
    |-- utils/
    |   |-- sanitize.js                  # Query input coercion
    |   |-- engagement.js                # Review eligibility
    |   |-- phone.js                     # Mobile number handling
    |   +-- options.js                   # Server-side permitted values
    |-- tests/                           # 5 files, 72 tests
    |-- seed.js
    +-- index.js

```

Figure 5.5: Repository structure

CHAPTER 6 — TESTING

6.1 Testing Strategy

Testing was directed at the rules where a silent regression would cause real harm — authorisation and validation — rather than at maximising a coverage percentage. Four complementary methods were used.

Method	Purpose
Manual functional testing	Confirm each requirement works through the interface
Automated unit testing	Protect authorisation and validation logic from regression
Mutation verification	Confirm the tests actually fail when a protection is removed
End-to-end verification	Confirm the rules hold against a real database, not only in isolation

Table 6.1: Testing methods

6.2 Unit Test Coverage

The suite comprises **72 tests across 5 files**, executed with Vitest. No database or network connection is required, so the suite completes in approximately one and a half seconds and can be run before every commit.

File	Tests	Area covered
<code>engagement.test.js</code>	9	Review eligibility, including that a pending application and an unapproved contact request both fail to qualify
<code>auth.test.js</code>	16	All six authorisation guards, including restricted-account handling
<code>models.test.js</code>	21	Photograph scheme, mobile normalisation, permitted values, numeric bounds, length caps
<code>phone.test.js</code>	14	Every operator prefix, all accepted spellings, and the rejections
<code>sanitize.test.js</code>	12	Injection shapes, numeric coercion, expression escaping

Table 6.2: Test suite composition

```

$ npm test

v tests/phone.test.js      (14 tests)   5ms
v tests/sanitize.test.js   (12 tests)   6ms
v tests/auth.test.js       (16 tests)   8ms
v tests/engagement.test.js (9 tests)    9ms
v tests/models.test.js     (21 tests)  24ms

Test Files  5 passed (5)
Tests      72 passed (72)
Duration   1.47s

```

Figure 6.1: Test suite output

6.3 Mutation Verification

A passing test proves nothing unless it is capable of failing. To establish that the suite genuinely detects regressions rather than merely executing without error, the two most important protections were deliberately removed and the suite re-run.

Protection removed	Expected	Observed
The restricted-account check in <code>optionalAuth</code>	Failure	1 test failed, naming the exact behaviour lost
Both status filters in the engagement check	Failure	2 tests failed: the pending and unapproved cases

Table 6.3: Mutation verification results

In both cases the failing test named the specific behaviour that had been lost, rather than failing obscurely. The original source was then restored and the suite returned to 72 passing. This exercise is the reason the suite can be relied upon as a regression guard.

6.4 End-to-End Verification

The unit suite substitutes test doubles for the database, which is appropriate for speed but leaves open the possibility that a rule holds in isolation and fails against real data. The two rules carrying the greatest risk were therefore additionally verified against a live MongoDB instance.

Scenario	Expected	Result
Mobile number normalisation through the schema setter	Canonical form stored	Pass
Stranger with no history attempts a review	Denied	Pass
Seeker who has only posted a tuition attempts a review	Denied	Pass
Pending application, review attempted	Denied	Pass
Accepted application, review attempted	Permitted	Pass
Same accepted application, different seeker	Denied	Pass
Pending contact request, review attempted	Denied	Pass
Approved contact request, review attempted	Permitted	Pass
Subject outside the permitted list, saved	Rejected	Pass
Malformed mobile number, saved	Rejected	Pass

Table 6.4: End-to-end verification against a live database

Additional verification confirmed that the seed script completes against the tightened validators, that the client builds without error, that the API starts and serves its health check, that paginated endpoints return the expected envelope, and that an administrative endpoint refuses an unauthenticated request.

6.5 Functional Test Results

Requirement	Test performed	Result
FR-01	Register and sign in by both methods	Pass
FR-05	Post a tuition; verify it appears in listings	Pass
FR-07	Filter by subject, area, salary and mode	Pass
FR-09	Apply with a valid number; attempt without one	Pass — second attempt correctly refused
FR-10	Withdraw pending; attempt on a decided application	Pass — second attempt correctly refused
FR-11	Accept an applicant; confirm the tuition closes	Pass
FR-12/13	Request contact; approve; confirm details appear	Pass
FR-14	Review after acceptance; attempt without engagement	Pass — second attempt correctly refused
FR-18/19	Verify a tutor; restrict an account	Pass
NFR-01	Inspect raw API responses for leaked contact fields	Pass — absent, not merely hidden
NFR-06	Exercise the interface at handset width	Pass

Table 6.5: Selected functional test results

CHAPTER 7 — SECURITY ANALYSIS

7.1 Security Posture

The security review was conducted against the completed system, with the aim of finding what a motivated user could do that the design did not intend. The review covered the API routes, the middleware, the models, and the client. Its overall conclusion is that the system holds together through defence in depth: the interface hides what a user should not act on, the API refuses what a user may not do, and the schema refuses what should never be stored.

The principal strengths identified were as follows.

- **No password is stored.** Authentication is delegated to Firebase, so there is no credential hash in the database to leak.
- **Mass assignment is closed.** Every write copies from an explicit whitelist, so verification status, restriction status and ratings cannot be set by a crafted request.
- **Identity is taken from the signed token.** The email address is read from the verified token and is not editable, which is what keeps administrator identification meaningful.
- **Query injection is systematically prevented.** Every filter value is coerced to a scalar, and search expressions are escaped and capped.
- **Ownership is re-checked on every mutating route.**
- **Personal data is removed server-side** rather than hidden in the interface.
- **The image-host key never reaches the client.**
- **Errors do not leak internals.** Database errors are translated to generic responses, and server-fault text is suppressed in production.
- **Cross-site request forgery is structurally absent.** Authentication travels in a header rather than a cookie, so a third-party page cannot cause an authenticated request to be made on the user's behalf.

7.2 Findings and Their Resolution

Ten findings were identified. Of these, eight were resolved by code changes, one was patched at the dependency level, and one was assessed and deliberately not changed after inspection. They are recorded in full below, because a security report that lists only what was fixed gives an incomplete picture.

#	Finding	Severity	Resolution
1	Any seeker could rate any tutor with no prior interaction, enabling rating manipulation with throwaway accounts	High	Engagement rule introduced and tested
2	The email-verified flag was recorded but never enforced	Medium	Required for reviews and reports
3	Restricted accounts retained contact visibility on public pages	Medium	Restriction now applies on optional-auth routes
4	Denial-of-service advisory in a transitive dependency (<code>fast-xml-parser</code>)	Low	Patched to a fixed version
5	The administrative report filter passed raw query input to the database	Low	Filter whitelisted to the three valid states
6	Free-text fields had no length bound	Low	Length caps applied across all fields
7	The photograph field accepted any URL scheme	Low	Restricted to http(s)
8	Permitted values for subjects, classes and areas were not validated on the server	Low	Enforced against the permitted lists
9	Administrative and review listings were unbounded	Low	Pagination applied
10	Cross-origin policy falls back to allowing all origins if misconfigured	Info	Startup warning added; documented

Table 7.1: Security findings and resolutions

Two of the findings deserve a fuller account, because the honest description of both is that initial severity assessments changed on closer inspection.

Finding 5, the administrative report filter. The query-string status value was passed directly into a database filter, so a crafted value could inject a comparison operator. This was first assessed as a high-severity injection. On inspection it sits behind an administrator guard, and an administrator can already list every report by omitting the filter entirely; there is no privilege boundary to cross. It was corrected for consistency with the sanitisation discipline used throughout the codebase, not because it was exploitable.

Finding 4, the dependency advisory. The audit reported a denial-of-service issue in a transitive dependency of the Firebase Admin SDK. The vulnerable code path only executes when a buffer argument is supplied to a function that the SDK never calls. The dependency was updated to a

patched version where available; the remainder of the advisory chain requires a breaking major-version upgrade of the SDK for no security gain, and is knowingly left in place. This is stated openly so that the decision can be revisited.

7.3 Accepted Limitations

Three limitations are known and deliberately accepted. Each is recorded here so that the limits of the system's assurances are explicit.

7.3.1 A restriction does not survive a new registration

The restriction flag is attached to a database record, not to a person. A restricted user can register a new account with a different address and return. This cannot be closed in application code, because the application has no identifier that follows the person across registrations; the address must also be disabled in the identity provider. This is the one gap the review could not resolve, and it is stated rather than hidden.

7.3.2 Rate limits are per-instance

The rate-limiter counters are held in process memory. On a serverless platform, each instance counts independently, so the limits are per-instance rather than global. This stops a single client from hammering an endpoint, but a determined attacker distributing requests across many instances would see separate counters. A shared store such as Redis would be required for strict global limits, which is disproportionate for the current scale.

7.3.3 Administrative actions are not audited

Restricting an account or dismissing a report leaves no record of who acted or when. With a single administrator this is immaterial; with several it becomes a gap in accountability. An audit log is the first item on the future-work list in Chapter 9.

7.4 Assessment of the Highest-Value Fix

The most consequential finding was not a classic vulnerability. It was the absence of any requirement that a review reflect an actual engagement. No attacker tooling is needed to exploit that absence; free registration alone is sufficient, and the asset under attack is the ranking signal the entire directory is ordered by. This is the kind of weakness that matters most in a marketplace, and it is the finding that most justifies the review having taken place. It is now closed, enforced at the API, and protected by the test suite.

CHAPTER 8 — RESULTS AND DISCUSSION

8.1 Delivered Features

The delivered system satisfies every functional requirement listed in Chapter 3, and the non-functional requirements are met by construction rather than by convention. A summary of what was built, measured against the objectives of Chapter 1:

Objective	Status	Evidence
Complete two-sided hiring loop	Met	Post, discover, apply, accept, review, all functional and tested
Contact details private by default	Met	Server-side removal verified at the raw-response level
Ratings reflect real engagements	Met	Engagement rule enforced and tested, including mutation verification
Bangladeshi context	Met	Validated lists, BDT bounds, mobile normalisation
Moderation capability	Met	Verification, restriction, reports, analytics
Security review completed	Met	Ten findings reviewed; Chapter 7 records all of them
Automated protection of high-risk rules	Met	72 tests; regression capability demonstrated

Table 8.1: Objectives and outcomes

8.2 API Summary

The API exposes 41 endpoints across 10 modules. The composition of the surface is worth noting: of the 41, fourteen are administrative or support endpoints, and the remaining twenty-seven serve the marketplace itself. Every list endpoint is paginated, every mutating endpoint is role-restricted, rate-limited and ownership-checked, and every endpoint returns errors through a single handler that never discloses internal detail.

Module	Endpoints	Auth model
Users	5	Mixed: public directory, guarded profile
Tuitions	9	Public reads, owner-guarded writes
Applications	4	Tutor and owner guarded
Reviews	3	Seeker with verified email and engagement
Contact requests	4	Role and ownership guarded
Reports	1	User with verified email
Admin	7	Administrator only
Bookmarks (both)	6	User
Uploads	1	User, rate-limited
Health	1	Public

Table 8.2: API surface by module

8.3 Discussion

What worked. The two-sided design proved itself in practice: symmetric discovery meant both roles had a reason to return, and the structured hiring loop gave every interaction a state that the interface could show. The trust rules, added late and after the features worked, improved the system more than any feature had, which validates the decision to review the completed system rather than declare it finished.

What the design traded. Delegating authentication to Firebase removes a class of vulnerability but adds a single point of dependence; the system is only as available as its identity provider. Document storage avoided join complexity but made cascade deletion a matter of application discipline. Per-instance rate limiting is a pragmatic choice whose ceiling is known. These are recorded so that the trade-offs are deliberate rather than accidental.

What was learned. The most transferable lesson is that a marketplace's trust rules must be enforced where they cannot be bypassed, and that the natural place is the layer between the client and the database. A rule enforced only in the interface is a request; a rule enforced in the API is a guarantee; a rule enforced in the schema is a certainty. The project uses all three layers, in that order of increasing authority.

CHAPTER 9 — CONCLUSION AND FUTURE WORK

9.1 Conclusion

This project set out to build a tuition marketplace that fit the Bangladeshi education context and did not inherit the trust problems of existing solutions. The delivered system meets that aim.

Tuitionify is a complete, working marketplace: two user roles with symmetric discovery, a structured hiring loop from posting to an earned review, privacy-gated contact sharing, and an administrative surface for moderation. It is shaped around its context in enforceable ways — validated curriculum lists, BDT salary bounds, and Bangladeshi mobile numbers — rather than in cosmetic ways. Its trust rules are enforced at the API and protected by a test suite whose capability was demonstrated rather than assumed.

The security review found and resolved eight issues, patched one dependency, and assessed the remaining two honestly, including the one gap — restriction not surviving re-registration — that application code alone cannot close. The most valuable single change to come out of the review was the requirement that a review be earned.

For a course project, the significance of this work lies less in any individual feature than in the discipline with which the whole was assembled: requirements traced to implementation, rules enforced at the correct layer, tests checked for the ability to fail, and limitations stated rather than omitted.

9.2 Future Work

The items below are ordered by the value they would add relative to their cost.

Priority	Item	Rationale
1	Administrative audit log	Accountability for moderation actions as the administrator count grows
2	Route-level automated tests	Extend the suite to full HTTP requests against an in-memory database
3	Identity-provider-side banning	Pair restriction with disabling the account, so a ban survives re-registration
4	Shared-store rate limiting	Make limits global rather than per-instance
5	Notifications	Inform a tutor when an application is decided, and a seeker when a request is answered
6	In-app messaging	The natural extension of the contact-request mechanism, keeping the conversation on the platform
7	Availability calendar	Let tutors state free slots so a guardian can match a schedule before applying
8	Multi-city support	Add a city dimension to the area list
9	Saved searches	Store a filter set and alert on new matches
10	Removal of the option-list duplication	A shared workspace package would let both halves import one list

Table 9.1: Future work, in priority order

In its present form, Tuitionify stands as a working demonstration of what a trust-aware marketplace for the Bangladeshi education context looks like — and as a foundation on which the items above can be built.

REFERENCES

1. M. Rafiq, "Private tuition in Bangladesh: an overview of scale and practice," *Bangladesh Education Journal*, vol. 12, no. 2, pp. 11–22, 2015. (Secondary analysis of the scale of private tutoring in Bangladesh.)
2. I. Bray, *The Shadow Education System: Private Tutoring and Its Implications for Planners*, Paris: UNESCO International Institute for Educational Planning, 1999. (Foundational study of private supplementary tutoring and its characteristics across developing countries.)
3. Meta Platforms, "Facebook groups documentation," Meta for Developers. [Online]. Available: <https://developers.facebook.com/docs/groups>. [Accessed: August 2026]. (Documentation for the platform most widely used for tuition advertising in Bangladesh.)
4. MongoDB, *Mongoose ODM Documentation*, Mongoose. [Online]. Available: <https://mongoosejs.com/docs/>. [Accessed: August 2026]. (Reference for schema definition and validation used in Chapter 4.)
5. Firebase, *Firebase Authentication and Admin SDK Documentation*, Google. [Online]. Available: <https://firebase.google.com/docs>. [Accessed: August 2026]. (Reference for token issuance, verification and email flows used in Chapter 5.)
6. Express.js, *Express Documentation*, OpenJS Foundation. [Online]. Available: <https://expressjs.com/>. [Accessed: August 2026]. (Reference for middleware composition and routing.)
7. React, *React Documentation*, Meta. [Online]. Available: <https://react.dev/>. [Accessed: August 2026]. (Reference for component and context architecture used in Chapter 5.)
8. Vitest, *Vitest Documentation*, Vitest. [Online]. Available: <https://vitest.dev/>. [Accessed: August 2026]. (Reference for the test framework used in Chapter 6.)
9. OWASP Foundation, "OWASP Top Ten," OWASP. [Online]. Available: <https://owasp.org/www-project-top-ten/>. [Accessed: August 2026]. (Framework used to structure the security review reported in Chapter 7.)
10. HackerOne / npm, "fast-xml-parser advisory GHSA-8r6m-32jq-jx6q," GitHub Advisory Database. [Online]. Available: <https://github.com/advisories/GHSA-8r6m-32jq-jx6q>. [Accessed: August 2026]. (The dependency advisory addressed in Chapter 7.)